Semester Project: Electronic Web Store

Web Programming Course - Semester Project Specification

Noa RISPAL
Student ID: 2560118
Domain: D4 - Electronics Store

Contents

1 Introduction **3**
- 1.1 GitHub . . . 3
- 1.2 Project Overview . . . 3
  - 1.2.1 Purpose . . . 3
  - 1.2.2 Website Purpose from a Business Perspective . . . 3

2 Domain-Specific Design Choices **3**
- 2.1 Domain Assignment . . . 3
- 2.2 Product Catalog Design . . . 3

3 Layout and Responsive Design **3**
- 3.1 Architecture . . . 3
- 3.2 Responsiveness . . . 4

4 Database Requirements **4**
- 4.1 Schema Design . . . 4
- 4.2 Entity-Relationship Diagram . . . 5

5 Implementation Details **5**
- 5.1 AJAX Search Functionality . . . 5
- 5.2 User Authentication . . . 5
- 5.3 SEO Best Practices . . . 6
- 5.4 Store Locations and Google Maps . . . 6

6 Challenges and Lessons Learned **6**
- 6.1 Challenges . . . 6
- 6.2 Lessons Learned . . . 7

7 Conclusion **7**

1 Introduction

1.1 GitHub

Repository Link: https://github.com/NoaRispal/electronicShop

1.2 Project Overview

This report details the design and implementation of a dynamic, database-driven electronics store website. Developed as part of the Web Programming course, this individual project integrates core web technologies to provide a professional user experience.

1.2.1 Purpose

The primary objective is to demonstrate proficiency in building a complete full-stack application, covering responsive front-end design, secure server-side logic with PHP, and efficient data management using MySQ.

1.2.2 Website Purpose from a Business Perspective

The Electronic Store Website is focused on user experience. The goal is not particularly to increase sales via online shopping but to encourage users to visit the different stores by assuring them that the locations have the right items in stock. Stores are intended to be used by local people, for whom visiting in person is faster than waiting for delivery.

2 Domain-Specific Design Choices

2.1 Domain Assignment

Following the project formula (Student ID $2560118 \pmod 5 = 3$), the assigned domain is **D4: Electronics Store**.

2.2 Product Catalog Design

Electronic products require technical specificity. The database and UI were designed to handle attributes such as technical specifications and multi-store availability. The visual theme uses a clean, professional aesthetic (Via Bootstrap) so that the user can see with one glance what each button does. In that order, I reused commonly used buttons, such as the cart or profile ones, and I use carousels and delimited cards to clearly show each product and its important data in one glance. The design was made by me (not from Figma).

3 Layout and Responsive Design

3.1 Architecture

The website follows a standard architectural pattern with the following components:

- **Header:** Contains the company logo and authentication status.
- **Navigation Bar:** Includes links for Home, Products, Search, and Contact.
- **Breadcrumbs:** Implemented as *Home* > *Products* > *Category* > *Item* for improved navigation.
- **Footer:** Displays copyright information and contact details.

3.2 Responsiveness

Responsive design was achieved using the **Bootstrap framework**. The layout adapts seamlessly across:

- **Desktop:** Grid-based catalog display.
- **Tablet:** Collapsible navigation and stacked elements.
- **Mobile:** Touch-friendly buttons and simplified navigation.

4 Database Requirements

4.1 Schema Design

The MySQL database consists of five core tables designed for data integrity:

- **Users:** Manages authentication and user profiles.
- **Products:** Stores item names, prices, and descriptions.
- **Categories:** Organizes products into logical groups.
- **Stores:** Lists physical locations of the shop.
- **Inventory:** A many-to-many junction table linking products to stores to track local availability.

All components communicate with the database through the Transaction class. Even if right now this class adds few features compared to a standard PDO object, the Transaction class will be needed if we want to optimize or add more security.

4.2 Entity-Relationship Diagram

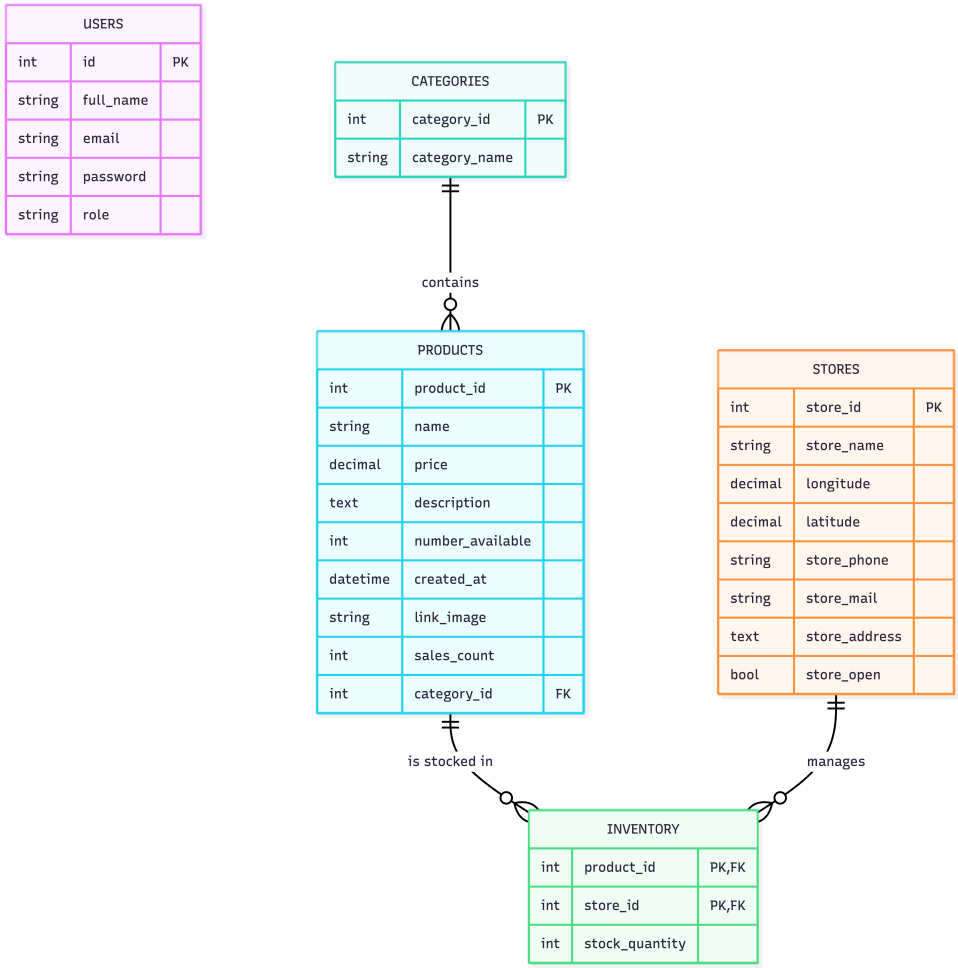

Figure 1: Entity-Relationship Diagram of the Electronic Shop

5 Implementation Details

5.1 AJAX Search Functionality

To comply with the requirement of a dynamic search without page reloads, AJAX is used. The client-side script sends asynchronous requests to a PHP handler as the user types, updating the DOM in real-time. While the user is typing, a small dropdown with the three best "alike" item names matching the search query is shown (these items are retrieved from the database). When the user clicks on the search button, they are redirected to the products page with filters linked to their search query applied. N.B. : The more items the database has, the less efficient this method is. We could implement a cache or other optimization methods to speed it up, but for this demo project, these simple requests to the database are enough.

5.2 User Authentication

The system implements secure registration and login:

- **Input Validation:** Includes client-side and server-side checks for email formats and password length.
- **Security:** Passwords are never stored in plain text; PHP's `password_hash()` function is utilized to hash and salt them.
- **Change Password:** The user can change their password using only their email address. In a real-life scenario, this would be a major security breach, and we would implement another controller to send an email and wait for confirmation from the user (this is not implemented here).

5.3 SEO Best Practices

Three specific SEO techniques were applied:

1. **Meaningful Metadata:** Dynamic page titles and meta-descriptions are included in the header. For this to work, the header must be included after the controller retrieves the data. To ensure that, I include the header at the beginning of each view file.
2. **Friendly URLs:** Implemented using `.htaccess` rewriting (e.g., `/products/laptop` instead of `?id=5`.
3. **Sitemap:** A dedicated `sitemap.php` file provides a structural map for search engines. It shows XML data for the localization, importance score, and update life cycle of every pages (product, category, and the homepage).

5.4 Store Locations and Google Maps

Each product page checks the **Inventory** table. If an item is available in a specific store, a dynamic link to Google Maps is generated using `urlencode()` for the store's address.

On the contact page, an interactive map is used to show the location and details of each store in HCMC. It uses the library Leaflet.js and OpenStreetMap to achieve this (map.js for the code).

6 Challenges and Lessons Learned

6.1 Challenges

The development process presented several technical hurdles that required iterative problem-solving:

- **AJAX Search Implementation:** One of my biggest challenges was making the AJAX search work smoothly. I had to make sure the UI stayed responsive while the script was fetching data from the MySQL database in the background. I also had to manage "Race Conditions"—where an old search result could return after a newer one—so I had to carefully handle the JavaScript `fetch()` API to ensure the DOM always showed the most recent input without any page reloads. Another tricky part was the CSS: the search dropdown initially pushed other navbar elements around, so I had to fix the styling to make it overlay correctly.

- **Store Locations and Availability:** Implementing this requirement was a challenge because it needed a proper database normalization. I had to create an `inventory` junction table to link `products` and `stores`, which was essential to avoid data redundancy. The main difficulty was writing the SQL JOIN queries: I needed to retrieve all the product details while simultaneously checking real-time availability across different physical locations, all while keeping the referential integrity of the database.

- **Friendly URLs:** Some of my categories have names with special characters (e.g., "Video & Music"). With friendly URLs, when controllers needed to retrieve data like the category, they were only able to retrieve "Video" because the system thought that after the "&" it was another parameter. Even with `urlencode()`, it didn't solve this problem. The solution came from the `.htaccess` file where, in my rewrite rule, I needed to add the escape backreferences flag `[B]`.

6.2 Lessons Learned

This project served as a practical application of modern web development principles:

- **Modular Software Architecture:** The project reinforced the necessity of the Model-View-Controller (MVC) pattern. Separating logic (controllers), presentation (views), and configuration (database) made the codebase significantly easier to debug and scale.

- **Defensive Programming with PDO:** Using PHP Data Objects (PDO) for all database interactions highlighted the importance of prepared statements. This practice not only made the code more readable but provided a robust defense against SQL injection attacks, ensuring the security of the User Authentication system.

- **The Value of SEO and UX:** Implementing "Friendly URLs" and semantic HTML demonstrated how small back-end changes can drastically improve a website's visibility and user experience.

7 Conclusion

The Electronic Web Shop fulfills all semester project requirements. It provides a functional, secure, and user-friendly experience that demonstrates a comprehensive understanding of both front-end and back-end web development.